DETAILED PROJECT DOCUMENTATION

Intelligent Campus Resources Optimization System

Review 2 Documentation — Development, Application and Measured Results

Field	Details
Course	Data Structure and Algorithms - II
Course Code	CCSE0301
Programme / Branch	B.Tech CSE-A, NIET Greater Noida
Faculty	Mr. Shamshad Ali
Assignment Type	Individual Assignment / PBL
Student	Aarohi Yadav
ERP / Roll No.	0251cse364 / 2501330100003
Project	Intelligent Campus Resources Optimization System
SDG	SDG 4 — Quality Education
Review	Review 2 — Development & Application
Review Coverage	Trees and Heaps, Graphs, Dynamic Programming, Backtracking and Branch and Bound
Current Progress	65%
Status	Working Python prototype implemented, tested and benchmarked on fictional sample data and synthetic data; real campus data, user interface and recurring bookings not yet done
Implementation	Python 3 (standard library only); repository: github.com/Aarohi591/intelligent-campus-resources-optimization-system-DSA

Note: This document is intentionally more detailed than the 3-page progress report. It is designed to serve as supporting evidence for the work reported in Review 2. The distinction between what is implemented and tested, what is only studied, and what is planned is maintained throughout. **All campus data in this project is fictional sample data and all benchmark input is synthetic; no real NIET campus data has been used.**

Table of Contents

1. Introduction 3
2. Project Domain and Context 3
3. Problem Statement 4
4. Problem Understanding and Identification 4
5. Existing Practice and Background 5
6. Literature Review — Two Selected Papers 6
7. Research Gaps Identified 6
8. Project Objectives 7
9. Stakeholders and User Requirements 7
10. Functional and Data Requirements 8
11. DSA-II Concepts: Study, Selection, Implementation and Rejection Analysis 9
12. Detailed Tree-Based Analysis 15
13. Detailed Graph-Based Analysis 16
14. Detailed Dynamic Programming Analysis 17
15. Allocation Method and Explanations 18
16. End-to-End System Workflow 18
17. System Architecture and Data Model 19
18. Algorithms, Code and Pseudocode 20
19. Complexity and Performance Considerations 21
20. Worked Example: The Sample Campus 23
21. Testing and Verification 24
22. Work Completed During Month 2 25
23. Evidence and Supporting Material 26
24. Challenges Faced and How They Were Addressed 26
25. Individual Contribution and Tools 27
26. Current Status and Progress 27
27. Plan for Final Review 27
28. Limitations 28
29. Conclusion 28
30. References 28

1. Introduction

A college runs on shared resources: classrooms, laboratories, seminar halls, projectors, grounds and courts. Far more people want to use them than can be served at once, and every resource can serve only one booking at a time. As the number of booking requests grows, the central challenge is no longer only recording who booked what; it is finding, quickly and fairly, which requests can be accepted together without any clash, and explaining why the others cannot.

The Intelligent Campus Resources Optimization System studies this problem from a Data Structures and Algorithms perspective. Instead of treating the system as a conventional booking form with a register behind it, the project focuses on the algorithmic layer: storing the resource inventory in a balanced tree, checking availability against per-resource calendars, modelling clashes as a graph, prioritising competing requests with a heap, choosing the most valuable set of bookings with dynamic programming, and mapping flexible bookings to real rooms using graph colouring and backtracking with branch and bound.

Review 1 was analytical: it fixed the problem, reviewed two research papers and matched Unit 1 and Unit 2 topics to the problem, with no code. **Review 2, documented here, is the implementation stage.** A working Python prototype now exists, with an automated test suite of 59 checks, a demonstration on fictional sample data, and a benchmark on synthetic data. No claim is made that real campus data has been used, that a user interface exists, or that the prototype is ready for deployment.

This documentation answers four questions:

- Why is conflict-free campus resource allocation a meaningful problem?
- What did existing practice and the literature show?
- Why were particular DSA structures selected or rejected?
- How do the selected structures fit together in the implemented prototype, and how did they perform?

2. Project Domain and Context

2.1 Domain

Domain: campus resource management and algorithmic, conflict-free booking and allocation of shared physical resources.

2.2 Why this domain was selected

The domain naturally contains several kinds of records and relationships. A resource has a capacity, equipment, opening hours and maintenance periods; a request has a time span, a priority, a size and equipment needs; and two requests can clash only if they want the same physical resource at overlapping times. Therefore the domain provides a practical setting in which trees, heaps, graphs, dynamic programming and backtracking each have a genuine job, without artificially adding unrelated structures.

Entity	Important attributes	Why it matters
Resource	ID, type, capacity, availability, equipment, daily opening hours, maintenance periods	The thing being allocated; its constraints decide which requests are suitable.
Booking request	ID, requester, department, resource type, start, end, priority, required capacity, required equipment, preferred resource	Source of the demand; priority is the value the system tries to maximise.
Requester / department	Faculty member, club or committee, and its department	Identifies who is served and appears in the explanation of every decision.
Calendar (schedule)	Sorted accepted bookings for each physical resource	Makes the availability check fast and prevents double booking.
Equipment	Names such as projector, microphone, computers, floodlights	A request can be unsuitable for a room even when time and size fit.

3. Problem Statement

The system addresses the problem of allocating shared campus resources to many competing booking requests without any double booking. Registers, spreadsheets and generic calendars already exist, so the problem is not simply the absence of a place to write bookings. The focus is on how resource and booking data can be organised and processed so that availability checking, clash detection, prioritisation and room assignment become systematic and algorithmically efficient.

The problem can be expressed as:

Given:

- A set of resources R1...Rm, each with a type, capacity, equipment, opening hours and maintenance periods
- A set of booking requests B1...Bn, each with a time span, a priority, a size, equipment needs and a resource type

Determine:

- Which resources could serve each request?
- Which requests clash with one another?
- Which set of requests should be accepted, and on which physical resource, so that the total priority is as high as possible?
- For each rejected request, what is the reason?

Output: a conflict-free schedule of accepted bookings, and a rejection with a reason for every request that is not accepted.

3.1 Why manual allocation becomes difficult

- Comparing every new request with every existing booking one at a time becomes far too slow as the number of requests grows (8,000 requests need 31,996,000 pair comparisons).
- Checking by eye in a register or spreadsheet misses clashes, especially partial overlaps such as 09:30-10:30 against 09:00-10:00.
- A free room is not necessarily a suitable room: it may be too small, lack a projector, be closed for maintenance or be outside its opening hours.
- When several requests want the same resource, deciding fairly needs a rule; the highest-priority request alone is not always the best choice for the campus.
- A bare "no" is not useful. Requesters need to know whether to change the time, the room type or the equipment.

4. Problem Understanding and Identification

4.1 Understanding developed during Review 1 and confirmed in Review 2

Area	Understanding
Existing problem	Checking bookings by eye and storing them in a flat list stop working once requests grow; clashes, capacity and equipment cannot be tracked reliably.
Users affected	Faculty, lab and facility in-charges, student clubs and event organisers, and college administration.
Core information	Resource ID, type, capacity, availability, equipment, requested time slot, requester and department, request priority.
Core operations	Look up a resource, check availability, detect clashes, prioritise, choose the best set, assign rooms, explain rejections.
Main DSA need	Ordered search of resource records (tree), clash modelling (graph), priority-based serving (heap), best-set selection (dynamic programming), room counting (backtracking and branch and bound).

Area	Understanding
Research direction	Combine interval storage, conflict-graph colouring and allocation-efficiency ideas into one workflow.

4.2 Problem formulation

The project treats campus allocation as a pipeline rather than one algorithm. Requests are first checked against the inventory, then clashes are modelled, then priorities are applied, then the best set is chosen, and finally rooms are assigned and every decision is explained.

```
Booking Requests + Resource Inventory
              |
              v
Eligibility Check            (availability, hours, capacity, equipment)
              |
              v
Conflict Graph               (adjacency list)
              |
              v
Priority Serving             (max-heap, best-fit free room)
              |
              v
Best Set per Resource        (dynamic programming)
              |
              v
Room Mapping                 (graph colouring, backtracking, branch and bound)
              |
              v
Accepted / Rejected + Reasons + Verification
```

5. Existing Practice and Background

During Review 1, the project was compared conceptually with how campus resources are commonly managed. The purpose was not to claim that existing tools are simple or lack features, but to define what is already common and where this academic project can focus its DSA contribution. The comparison below is general; no specific product is described or evaluated.

Aspect	Common practice	Project focus
Recording bookings	Register, shared spreadsheet or a generic calendar	A resource inventory and a per-resource calendar that refuses overlaps
Clash checking	By eye, or one request at a time	A conflict graph built by a sorted sweep; interval logic that is tested
Suitability	Left to the person booking	Capacity, equipment, opening hours and maintenance checked automatically
Competing requests	First come, first served, or negotiation	Priority-based serving, then dynamic programming for the best total value
Room choice	Manual	Best-fit choice, and colouring-based room mapping when valid
Explanations	Usually none	A reason code and message for every rejection
Academic contribution	Complex or proprietary scheduling software may exist	A transparent, explainable DSA-based implementation suitable for academic evaluation

Note: The purpose of this comparison is scope definition. The project does not claim to reproduce or outperform any commercial timetabling or booking software.

6. Literature Review — Two Selected Papers

Only the two papers discussed for Review 1 are included in this documentation.

6.1 Evaluating Graph Coloring Algorithms for University Course Timetabling: Welch-Powell and Backtracking Approaches (2025)

This paper uses graph-colouring techniques to model university courses, classrooms and scheduling constraints so that conflicts are avoided. It compares a greedy Welch-Powell colouring with a backtracking approach.

Relevance to this project: It is the closest conceptual foundation for the clash graph. Both colouring methods are implemented here and compared on the booking conflict graph. The project's extension is to apply colouring only to bookings that can use more than one room, and to map colours to real rooms only when the room is valid for every booking in the colour class.

6.2 The optimization model and empirical analysis of teaching resource allocation and business collaboration under the background of higher education informatization (2026)

This paper studies teaching resource allocation and management in higher education and develops an optimisation framework for evaluating the efficiency of resource allocation.

Relevance to this project: It supports treating allocation as an efficiency problem. This is why the allocation step compares a quick priority-based method with an optimal dynamic-programming method and reports the difference in total value.

6.3 Paper-to-project comparison

Paper	Focus	What we take from it
Graph colouring for course timetabling (2025)	Conflict-free timetabling by colouring; Welch-Powell and backtracking	Clash graph, colouring and backtracking methods
Teaching resource allocation optimisation (2026)	Efficiency of resource allocation in higher education	Allocation as an optimisation problem with a measurable value

7. Research Gaps Identified

The gaps below are stated carefully. Where a point is taken directly from the reviewed literature it is presented as a literature observation. Where the project proposes an extension based on the scope of a paper, it is explicitly treated as the project's proposed direction, not as an author-stated limitation.

Gap / Observation	Basis	How the project responds
Colouring-based timetabling and allocation-efficiency work are treated separately	Inferred from the scope of the two papers	Run eligibility, conflict graph, priority, DP and colouring in one workflow on the same data
Colouring alone does not decide who is accepted when demand exceeds supply	Derived from the project's required operations	Add priority-based serving and dynamic programming to choose the most valuable set
A colouring may assign a room that cannot actually host a booking	Derived from capacity and equipment constraints	Map colours to rooms only when one room suits every booking in the class; otherwise keep the existing assignment
Users need reasons, not only results	Project design need	Return a reason code and message for every rejection
Efficiency claims need measurement	Project design need; allocation-efficiency paper	Test with 59 checks and benchmark on synthetic data, reporting counts and values

7.1 Core project gap statement

The project explores how conflict-graph colouring and allocation-efficiency ideas can be combined into a transparent campus booking workflow that stores the resource inventory in a balanced tree, checks availability efficiently, prioritises competing requests, chooses the most valuable conflict-free set, and explains every rejection.

7.2 Important viva clarification

The research papers do not prescribe an AVL tree, a max-heap or the dynamic-programming recurrence used here. Those structures are selected by the project based on the required operations. The colouring and backtracking methods do come from the first paper. This distinction is important: literature motivates the problem and the research direction, while DSA analysis determines the implementation structures.

8. Project Objectives

1. Understand how campus resources can be allocated to requests without clashes.
2. Understand why checking bookings by eye and storing them in a flat list stop working as requests grow.
3. Study DSA-II Units 1 to 4 and determine which concepts are genuinely applicable.
4. Use a balanced tree (AVL) for efficient ordered search of resource records, with a plain BST as a baseline.
5. Use a graph with an adjacency list to model clashes between competing requests.
6. Use a max-heap to prioritise competing requests, and dynamic programming to maximise the total value of accepted bookings.
7. Use graph colouring with backtracking and branch and bound to count and assign rooms, mapping to real rooms only when valid.
8. Implement, test and measure the design, and explain every rejection with a clear reason.

9. Stakeholders and User Requirements

Stakeholder	Need	Expected system support
Faculty members	Classrooms and labs available for scheduled classes without last-minute clashes	Suitable-room search, clash-free schedule, clear reasons if a request fails
Lab and facility in-charges	A clear record of which resource is booked, by whom and for how long	Per-resource calendars, schedule listing, maintenance periods and out-of-service status
Student clubs and event organisers	Halls, courts and equipment for events without colliding bookings	Equipment-aware matching, best-fit room choice, online request checking
College administration	A fair and quick process when many departments want the same resource	Priority-based allocation, value maximisation, verifiable schedule

9.1 Requester-side flow

1. The requester submits type, time span, size, equipment needs, priority and optionally a specific resource.
2. The system finds the resources that could serve the request, or rejects it at once with the reason.
3. If suitable resources exist, the request competes with other requests for them.
4. The system chooses the accepted set and assigns a physical resource to each accepted request.
5. The requester receives either the resource and time, or the reason for rejection.

10. Functional and Data Requirements

10.1 Functional requirements

- Maintain a resource inventory with capacity, availability, opening hours, maintenance periods and equipment.
- Accept booking requests.
- Check each request against capacity, equipment, availability and opening hours.
- Reject any booking that would overlap another booking on the same physical resource.
- Identify requests that conflict with one another.
- Prioritise competing requests and maximise the total value of accepted bookings.
- Assign accepted bookings to physical resources, and map flexible bookings to rooms only when valid.
- Explain every rejection.
- Re-check the finished schedule independently.
- Measure algorithmic performance and keep those results separate from campus results.

10.2 Resource record

```
Resource {
    id            # unique, e.g. "CR-101" (key of the AVL tree)
    type          # classroom, lab, seminar_hall, projector, sports_ground, indoor_court
    capacity      # people or seats
    available     # False = out of service
    equipment     # set of names, e.g. {"projector", "whiteboard"}
    name
    window        # (open, close) minutes of every day, or None
    blackouts     # tuple of (start, end) maintenance periods
}
```

10.3 Booking request record

```
BookingRequest {
    id, requester, dept
    res_type                # kind of resource wanted
    start, end              # minutes since Monday 00:00, half-open [start, end)
    priority                # integer 1..10, the value of the booking
    capacity                # seats needed
    equipment               # equipment the resource must have
    preferred_res           # specific resource ID, or None
    purpose
}
```

10.4 Why these attributes were selected

Attribute	Purpose in allocation
Type and capacity	Decide which resources can serve the request and support best-fit choice
Equipment	Reject rooms that cannot support the activity (for example no projector)
Availability, opening hours, maintenance	Reject resources that are closed or out of service
Start and end	Define the interval used for clash detection; half-open so back-to-back bookings are allowed
Priority	Order competing requests and define the value to maximise
Preferred resource	Lets a requester pin a booking to one room; it then has exactly one suitable resource

11. DSA-II Concepts: Study, Selection, Implementation and Rejection Analysis

This section explains exactly how the DSA-II concepts connect to the Intelligent Campus Resources Optimization System. The purpose is not to include every topic simply because it appears in the syllabus. Each structure is evaluated against a concrete system operation: storing records, checking availability, modelling clashes, prioritising requests, choosing the best set, or assigning rooms.

Important Review 2 status: every structure marked **Used** below is implemented in `campus.py` and exercised by the test suite. Structures marked **Not used** or **Considered** were studied and deliberately left out, and the reason is given. Unit 4 and Unit 5 slides were not part of the material reviewed for this document; Unit 4 is mapped by its syllabus title (backtracking, branch and bound), and no Unit 5 topic is applied.

Overall implemented pipeline: Requests and Inventory → Eligibility Check → Conflict Graph → Max-Heap Pass → Dynamic Programming → Colouring and Room Mapping → Fill Pass → Reasons and Verification.

11.1 DSA selection philosophy

The project needs three main capabilities. First, it must look up and check resources quickly. Second, it must represent which requests compete with which, because one request can compete with many others. Third, when demand exceeds supply it must choose a set of requests that is as valuable as possible, and assign rooms validly. Trees are used for ordered lookup, graphs for competition, heaps for priority, dynamic programming for the best set, and backtracking with branch and bound for exact room counts. Structures whose strengths do not match a core requirement are documented as not used or considered, rather than being artificially inserted.

11.2 Data that the structures process

The main records are Resource and BookingRequest. The same data is viewed in different ways: the AVL tree is an ordered index over resource IDs; each resource's calendar is a sorted list of its bookings; the conflict graph represents which requests could compete; the heap temporarily orders requests by priority; and the colouring works on the clash graph of one group of bookings.

11.3 Binary Tree — studied, not a core structure

What it is: A hierarchical structure in which each node has at most two children, with no ordering rule.

Why it is not used for the core inventory: Resources are looked up by ID. A plain binary tree has no ordering rule, so a lookup may need to visit nodes across the whole tree, which is O(n) in the worst case. The ordered BST and AVL tree provide directed search instead.

Where it still appears: The heap is a complete binary tree stored in an array, and BST and AVL are binary trees with extra rules. Other binary-tree types (strict, full, extended) have no role in allocation.

Final decision: Studied for understanding; NOT used as a core project structure.

11.4 Binary Search Tree (BST) — baseline implementation

What it is: A binary tree where keys smaller than a node go to the left subtree and larger keys to the right.

Why it is implemented: It directly solves ordered search and is the baseline that explains why balancing matters.

How it is implemented: BST in `campus.py`: iterative insertion that walks down comparing resource IDs, and search that counts comparisons. Deletion is not implemented because the prototype never removes a resource.

Measured behaviour: When 8,000 resource IDs are inserted in sorted order the BST degenerates into a chain: height 8,000 and an average of 4,000.5 comparisons per search.

Final decision: BST is used as the baseline ordered tree, to be compared with AVL.

11.5 AVL Tree — core balanced tree (resource inventory)

What it is: A self-balancing BST. After every insertion the balance factor (left height minus right height) of every node on the path is checked and must stay -1, 0 or +1.

Why it is implemented: The inventory should give predictable lookups whatever the insertion order. AVL guarantees O(log n) search and insertion.

How insertion works: (1) Normal BST insertion. (2) Update the height on the return path. (3) Compute the balance factor. (4) If it leaves the range, apply one of four rotations: LL → right rotation; RR → left rotation; LR → left rotation of the child then right rotation; RL → right rotation of the child then left rotation.

```
def _ins(self, x, r):
    if x is None: return AVL.Node(r)
    if r.id < x.r.id: x.l = self._ins(x.l, r)
    elif r.id > x.r.id: x.rt = self._ins(x.rt, r)
    else: return x
    self._upd(x)
    b = self.H(x.l) - self.H(x.rt)
    if b > 1:
        if r.id > x.l.r.id: x.l = self._rot_l(x.l)
        return self._rot_r(x)
    if b < -1:
        if r.id < x.rt.r.id: x.rt = self._rot_r(x.rt)
        return self._rot_l(x)
    return x
```

How it fits the system: `Inventory` stores every resource in an AVL tree keyed by resource ID. Lookups by ID, duplicate-ID checks and sorted listing all use it.

Measured behaviour: For 8,000 sorted IDs the AVL tree has height 13, averages 12.0 comparisons per search and performs 7,987 rotations. The tests confirm that the tree stays balanced and that its inorder listing is sorted.

Not implemented: AVL deletion; the prototype never removes a resource.

Final decision: AVL is a CORE implemented structure for the inventory.

11.6 Max-Heap and Priority Queue — core priority structure

What it is: A complete binary tree stored in an array in which every parent has a priority at least as high as its children, so the best element is always at the root.

Why it is implemented: When requests compete, the system must repeatedly take the highest-priority request. A max-heap supports this directly: peek O(1), push O(log n), pop O(log n).

How it is implemented: `RequestHeap`: sift-up on push and sift-down on pop. The ordering is priority, then earlier start, then smaller ID, so results are deterministic.

How it fits the system: The heap pass pops requests highest priority first and gives each the smallest suitable free room. The same heap is used again in the fill pass.

Not implemented: Increase-key / change-priority; the heap is rebuilt for each pass. A min-heap has no role because the objective is the highest priority, not the lowest.

Final decision: Max-Heap is a CORE implemented structure. Min-Heap is NOT used.

11.7 Tree traversal — supporting

Inorder traversal (left, node, right) of the AVL tree lists resource IDs in sorted order and is used by `AVL.ids()`, the demo's inventory table and the tests. Preorder, postorder, level-order and building a tree from traversals are not used. A full traversal is O(n).

Final decision: Inorder is a SUPPORTING implemented operation; other traversals are NOT used.

11.8 Threaded Binary Tree and Huffman coding — not used

Threaded Binary Tree: Stores links to inorder neighbours in otherwise empty pointers so inorder traversal needs no stack. The inventory is small and traversed rarely, so the extra complexity buys nothing.

Huffman coding: A greedy compression algorithm built on a tree. A booking system has nothing to compress.

Final decision: Both are intentionally NOT used.

11.9 Graph — core clash model

Why a graph is necessary: Competition between requests is many-to-many: one request can clash with several others, and each of those can clash with others. A tree, with one parent per node, cannot represent this.

Graph model: Vertices are suitable booking requests. An undirected edge joins two requests that overlap in time *and* have at least one suitable resource in common. A flexible request therefore does not clash with a request that could only use a different room.

How it is implemented: `ConflictGraph`. Requests of one type are sorted by start time and each is compared only with the following requests that start before it ends, so the build is O(n log n + E) instead of O(n^2). The result is identical to the all-pairs build (tested).

How it is used: It lists competing pairs in the demo, supplies the neighbours that explain a TIME_CLASH rejection, and (in a smaller form) is the graph that is coloured in Section 11.15.

Measured behaviour: For 8,000 synthetic requests the all-pairs method makes 31,996,000 comparisons and the sweep 15,933, both finding the same 7,934 clashes.

Final decision: Graph is a CORE implemented structure.

11.10 Adjacency List and Adjacency Matrix

Adjacency list — used: For each vertex, a list of the vertices it clashes with. The clash graph is sparse, so storage is O(V + E).

Adjacency matrix — considered and rejected: A V×V table needs O(V^2) space. For 8,000 requests that is 64 million cells to represent only 7,934 clashes. Direct edge lookup in O(1) is not needed because the algorithms iterate over neighbours.

Final decision: Adjacency List is CORE. Adjacency Matrix is NOT preferred; it could be a comparison in later work.

11.11 BFS, DFS and other graph algorithms — not used

BFS, DFS, connected components, minimum spanning trees (Kruskal, Prim), transitive closure (Warshall) and shortest paths (Dijkstra, Bellman-Ford, Floyd-Warshall) were studied in Unit 2 but are not applied. Clashes are found by a sorted sweep rather than graph search; independent groups of requests exist but are not extracted; and there is no cost network or travel-time model. If travel time between buildings is added in future work, the shortest-path algorithms would become relevant.

Final decision: NOT used in the Review 2 prototype.

11.12 Greedy algorithm — baseline and first pass

What it is: An algorithm that makes the locally best choice at each step.

How it is used: The heap pass is greedy: it takes requests highest priority first and accepts one if a suitable resource is free. This gives a valid schedule quickly.

Its weakness: Accepting the highest-priority request can block two smaller requests that together are worth more. In the sample data a priority-6 auditorium booking blocks two priority-5 bookings (total 10).

Final decision: Greedy is USED as the first pass and as the baseline against which DP is measured.

11.13 Dynamic Programming — core best-set selection

What it is: A technique for problems with optimal substructure: the best solution is built from the best solutions of smaller subproblems, stored in a table.

Which problem is solved: Weighted interval scheduling on one resource: choose bookings that do not overlap so that the total priority is maximal. Sorted by end time, with p(j) the number of bookings that end at or before booking j starts: dp[j] = max(dp[j-1], priority(j) + dp[p(j)]). The table is filled bottom-up (tabulation) and the chosen set is recovered by tracing back.

```
def weighted_interval_dp(q, idx, want_set=False):
    """Weighted interval scheduling on ONE resource.
    dp[j] = max(dp[j-1], w_j + dp[p(j)]), p(j) = number of bookings ending at or before start_j."""
    idx = sorted(idx, key=lambda i: q[i].end)
    ends = [q[i].end for i in idx]
    m = len(idx)
    p = [0] * (m + 1)
    for j in range(1, m + 1):
        p[j] = bisect_right(ends, q[idx[j - 1]].start, 0, j - 1)
    dp = [0] * (m + 1)
    for j in range(1, m + 1):
        dp[j] = max(dp[j - 1], q[idx[j - 1]].priority + dp[p[j]])
    chosen = []
    if want_set:
        j = m
        while j > 0:
            if q[idx[j - 1]].priority + dp[p[j]] == dp[j]:
                chosen.append(idx[j - 1]); j = p[j]
            else:
                j -= 1
    return dp[m], chosen
```

How it is applied to the campus: For every resource the candidates are its current bookings plus any unplaced request that could use it. If the DP finds a strictly higher total the resource is re-planned. The result is never worse than the heap pass.

What is guaranteed: When every request is pinned to one resource, the result equals the independent per-resource optimum (tested on 20 random campuses). When requests are flexible the method is a local improvement, not a guaranteed global optimum.

0/1 Knapsack — considered: Same family of DP, but each resource here serves one booking at a time. It would fit an equipment-hours budget (future work).

LCS and matrix chain multiplication — not used: No operation in booking maps to them.

Resource allocation problem as taught (M units among N projects) — considered: The project allocates time slots on one resource rather than units among projects. The unit-budget version is future work.

Memoization — not used: The bottom-up table was sufficient.

Final decision: Dynamic programming (weighted interval scheduling) is a CORE implemented technique.

11.14 Backtracking and Branch and Bound — core exact room counting

What it is: Backtracking builds a solution step by step and undoes a step when it cannot lead to a solution. Branch and bound adds a bound that cuts any branch that cannot beat the best solution found so far.

Which problem is solved: The minimum number of rooms needed by a group of flexible bookings equals the minimum number of colours needed to colour their clash graph.

How it is implemented: `exact_colouring` colours vertices in decreasing-degree order, trying each existing colour and then one new colour. With the bound enabled a branch is cut as soon as it already uses as many colours as the best complete colouring found.

Measured behaviour: At 12 requests on one resource, plain backtracking visits on average 1,035,812 nodes and branch and bound 37, with the same optimum. The exact method is used only for groups of up to 14 bookings.

Final decision: Backtracking and branch and bound are CORE implemented techniques. Other Unit 4 example problems are NOT used.

11.15 Graph colouring and room mapping — core

Welch-Powell: Colours vertices in decreasing order of degree, one colour class at a time. Fast but not always optimal (from the first reviewed paper).

Mapping colours to rooms: A colour class (bookings that never overlap in time) may be given one real room only if that room is suitable for every booking in the class and is free of every other accepted booking. If any class cannot be placed nothing changes, and a mapping is adopted only if it uses no more rooms than before.

```
def assign_colour_classes(classes, elig, base_cal):
    """Give every colour class (requests that never overlap in time) ONE physical room.
    Valid only if the room is suitable for every member and free of other bookings. Returns
    {request id: room} or None when any class cannot be placed (then nothing may be reassigned)."""
    cal, mapping = base_cal.copy(), {}
    for cls in sorted(classes, key=lambda c: (-len(c), str(c[0].id))):
        common = set(elig[cls[0].id])
        for r in cls[1:]: common &= set(elig[r.id])
        room = next((x for x in elig[cls[0].id] if x in common
                     and all(cal.blocking(x, r.start, r.end) is None for r in cls)), None)
        if room is None: return None
        for r in cls: cal.book(room, r.start, r.end, r.id); mapping[r.id] = room
    return mapping
```

Measured behaviour: Welch-Powell matched the exact optimum in all 390 random cases tried (8 to 40 requests on one resource), so on this data its advantage is speed rather than quality. On the sample campus the mapping keeps the same room counts, so it is a validity-checked re-packing, not a proven improvement.

Final decision: Colouring with validated room mapping is a CORE implemented technique.

11.16 Sorting

Sorting by start or end time is needed for the sweep, the calendar and the DP. Python's built-in sort is used. The sorting algorithms taught in the course were not re-implemented.

Final decision: Sorting is USED through the standard library only.

11.17 Single-threaded execution — initial choice, and multithreading as future work

The prototype runs sequentially: eligibility, graph building, heap pass, DP, colouring and verification run in a fixed order. This makes every stage independently testable and every result reproducible. Multithreading could later serve many simultaneous users, but it would bring locking and shared-state problems that are unrelated to the DSA goals. The calendar is not safe for concurrent use today.

Final decision: Single-threaded execution is SELECTED. Multithreading is FUTURE work.

11.18 Why the selected structures work together

The structures are not competing alternatives for the same operation. The AVL tree answers "which resource is this?"; each calendar answers "is this slot free?"; the graph answers "which requests compete?"; the heap answers "which request first?"; dynamic programming answers "which set is best on this resource?"; and colouring with branch and bound answers "how many rooms are really needed and which room for which booking?". Each has a specific input, operation, output and complexity, which makes the design easy to explain and test.

11.19 Concepts not implemented as core — final justification

Concept	Status	Reason
Binary tree (plain), strict/full/extended types	Not used	No ordered search; no role in allocation.
Threaded binary tree	Not used	Stack-free traversal is not a project requirement.
Huffman coding	Not used	Nothing to compress.
Min-heap, heap increase-key	Not used	The objective is highest priority; the heap is rebuilt per pass.
BST and AVL deletion	Not used	The prototype never removes a resource.
Preorder, postorder, tree from traversals	Not used	No rebuilding of the inventory from traversals.
Adjacency matrix	Considered	$O(V^2)$ space; 64 million cells for 7,934 clashes.
BFS, DFS, components, MST, transitive closure, shortest paths	Not used	No search, network-cost or route problem exists yet.
0/1 knapsack, resource allocation DP (units)	Considered	Future work for equipment-hour budgets.
LCS, matrix chain multiplication, memoization	Not used	No matching operation; tabulation was sufficient.
Unit 5 (Advanced Data Structures)	Not used	No topic from the unit is applied.
Multithreading	Future	Correctness and reproducibility first.

These are deliberate engineering decisions, not omissions. Documenting both selection and rejection keeps the DSA mapping technically justified.

11.20 Review 1 vs Review 2 implementation boundary

Completed in Review 1: problem understanding, two-paper literature review, gap identification, study of Units 1 and 2, DSA applicability analysis. No code. **Completed in Review 2:** the Python prototype with all Used structures, 59 automated checks, a demo on sample data and a benchmark on synthetic data. **Planned for the final review:** real data import, a user interface, extra booking rules and final evaluation (Section 27).

12. Detailed Tree-Based Analysis

12.1 Why a tree is considered

A campus has an inventory of resources that must be looked up by ID and checked for duplicates. A tree keeps such records in order so a lookup does not scan everything. The choice of tree matters: an ordinary binary tree has no search order, a plain BST can degenerate, and an AVL tree stays balanced.

12.2 BST and AVL compared on the same input

Resources (n)	BST height	AVL height	BST avg. comparisons	AVL avg. comparisons
1,000	1,000	10	500.5	9.0
2,000	2,000	11	1,000.5	10.0
4,000	4,000	12	2,000.5	11.0
8,000	8,000	13	4,000.5	12.0

Sorted resource IDs are the worst case for a BST. Source: benchmark on synthetic IDs, repeatable.

12.3 AVL operations

Operation	Purpose in the project	Complexity	Status
Search	Find a resource by ID	O(log n) guaranteed	Implemented
Insert	Add a resource to the inventory	O(log n) guaranteed	Implemented
Delete	Remove a resource	O(log n)	Not implemented
Inorder traversal	List resources in sorted order	O(n)	Implemented

12.4 Traversals

Traversal	Complexity	Potential role	Status
Inorder	O(n)	Sorted listing of the inventory	Used
Preorder	O(n)	Serialisation of a tree	Not used
Postorder	O(n)	Bottom-up processing or clean-up	Not used
Level-order	O(n)	Breadth-wise inspection	Not used

12.5 The heap as a tree

The priority queue is a complete binary tree stored in an array. For index i, the parent is (i-1)/2 and the children are 2i+1 and 2i+2, so no pointers are needed. Heap order is restored by moving an element up (push) or down (pop) along one root-to-leaf path, which is why both operations are O(log n).

13. Detailed Graph-Based Analysis

13.1 Why a graph is necessary

Competition between bookings is a many-to-many relationship. One request can clash with several others, and a flexible request can compete for any of several rooms. A graph represents this naturally; a tree, which allows only one parent per node, cannot.

13.2 The implemented graph model

```
Vertex : a booking request that has at least one suitable resource
Edge   : two requests overlap in time AND share a suitable resource
         (undirected, unweighted)

Example (sample data, Monday classrooms):
   #1 (09:00-10:00)  --- clash ---  #2 (09:00-11:00)
   #1 (09:00-10:00)  --- clash ---  #3 (09:30-10:30)
   #2 (09:00-11:00)  --- clash ---  #3 (09:30-10:30)
   #5 (needs 150 seats) has no suitable room, so it is not a vertex
```

13.3 Graph components

Component	Representation
Vertex	A suitable booking request (ID from the request)
Edge	Overlap in time plus at least one shared suitable resource
Weight	None; priorities are stored on the requests, not on edges
Adjacency list	For each vertex, the sorted list of clashing vertices

13.4 Adjacency list vs adjacency matrix

Aspect	Adjacency list	Adjacency matrix
Space	O(V + E)	O(V²)
Best suited for	Sparse graphs	Dense graphs
Campus fit	Preferred: 7,934 clashes among 8,000 requests	Wasteful: 64 million cells for the same graph
Edge check	O(degree)	O(1)
Neighbour iteration	O(degree)	O(V)

13.5 Colouring the clash graph

A proper colouring gives joined vertices different colours. When vertices are bookings and edges are time overlaps, each colour class is a set of bookings that never overlap, so a whole class can share one room, and the number of colours is the number of rooms needed. Welch-Powell gives a quick colouring; exact backtracking with a bound gives the minimum. For interval graphs such as these the minimum equals the largest number of bookings that are simultaneously active, which the tests use as an independent check.

14. Detailed Dynamic Programming Analysis

14.1 Why greedy is not enough

The sample auditorium receives a rehearsal (09:00-13:00, priority 6) and two talks (09:00-11:00 and 11:00-13:00, priority 5 each). The heap serves the highest priority first, takes the rehearsal and blocks both talks: value 6. The best choice is the two talks: value 10.

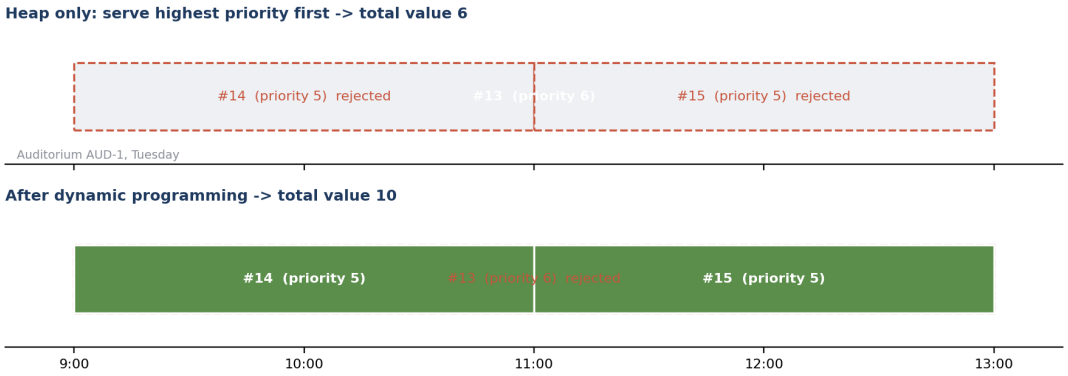


Figure 1 - Heap only (value 6) compared with dynamic programming (value 10) on the sample auditorium.

14.2 Recurrence and worked table

dp[0] = 0, dp[j] = max(dp[j-1], priority(j) + dp[p(j)]), where bookings are sorted by end time and p(j) is the number of bookings that end at or before booking j starts.

j	Booking	Interval	Priority	p(j)	dp[j]	Decision
1	#14 Alumni talk	09:00-11:00	5	0	5	take (5 + dp[0])
2	#13 Rehearsal	09:00-13:00	6	0	6	take (6 beats 5)
3	#15 Placement talk	11:00-13:00	5	1	10	take (5 + dp[1] = 10 beats 6)

Tracing back from dp[3] selects #15 and #14, total 10. The table has one entry per booking and each p(j) is found by binary search, so the cost is O(m log m) for m bookings on the resource.

14.3 Measured value of DP over heap-only (synthetic data)

Requests	Time slots	Heap-only value	DP value	DP gain
1,000	300	4,908.9	4,916.5	0.15%
2,000	300	8,899.5	8,948.8	0.55%
4,000	300	15,062.4	15,298.5	1.57%
4,000	100	9,582.5	10,225.5	6.71%
8,000	100	12,618.9	14,511.2	15.00%

Average of 20 seeds, 40 resources. Where requests are spread thin, greedy is nearly as good; on a busy campus DP matters.

14.4 Honest limits

The DP is exact for one resource. Across resources with flexible requests the method applies it per resource and only when the value rises, so it never lowers the value but is not proven globally optimal (that general problem is NP-hard). When every request is pinned to one resource the result equals the independent optimum.

15. Allocation Method and Explanations

15.1 Why a staged method

A single algorithm cannot handle suitability, competition, value and room assignment at once. Splitting the work into stages keeps each stage simple, testable and explainable, and each later stage can only improve the schedule or keep it unchanged.

15.2 Value model

The value of a schedule is the sum of the priorities of its accepted bookings. Priority is an integer from 1 to 10 supplied with the request. This is a simple, transparent value model; fairness between departments is not modelled (Section 28).

15.3 Explainability: reason codes

Code	Meaning	Example from the sample run
TIME_CLASH	Every suitable resource is already booked in that period	#9: both labs are taken Tue 14:00-17:00
CAPACITY	No resource is big enough	#5: CR-201 holds 120, needs 150
EQUIPMENT	No resource has the required equipment	#6: no classroom has a smartboard
UNAVAILABLE	Out of service or closed for maintenance	#11: Chemistry Lab out of service; #12: CS Lab 2 under maintenance
OUTSIDE_HOURS	Outside the resource's opening hours	#21: ground open 06:00-19:00, request 19:00-21:00
INVALID_TIME	End is not after start	Covered by tests
NO_RESOURCE_TYPE	No resource of that type in the inventory	Covered by tests
BAD_PREFERENCE	Preferred resource missing or of the wrong type	Covered by tests

When no resource is suitable, the reason comes from the *closest* candidate: fewest permanent mismatches (capacity, equipment), then fewest failures, then the largest capacity. That is why a request for a fume-hood lab reports that the Chemistry Lab is out of service instead of reporting that other labs lack a fume hood.

16. End-to-End System Workflow

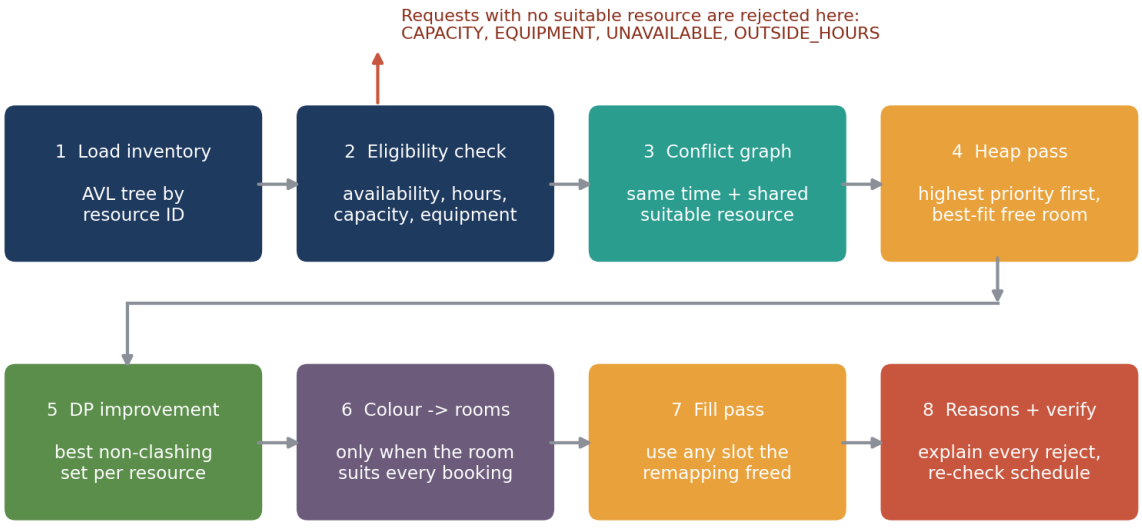


Figure 2 - The eight allocation stages of allocate_campus().

```
STEP 1   Load the inventory into an AVL tree (duplicate IDs refused)
   |
STEP 2   Check each request against every resource of its type; reject with a reason if none fits
   |
STEP 3   Build the conflict graph (overlap in time + shared suitable resource)
   |
STEP 4   Heap pass: highest priority first, smallest suitable free room
   |
STEP 5   DP improvement per resource (applied only if the value strictly rises)
   |
STEP 6   Colour flexible bookings; map colours to rooms only when valid
   |
STEP 7   Fill pass: offer freed slots to still-unplaced requests
   |
STEP 8   Explain every rejection; re-check the schedule with an independent verifier
```

16.1 Why the stages are separated

Stage	Reason for separation
Eligibility	Rejects impossible requests early and produces the most useful reason
Conflict graph	Captures which requests compete, for the explanation and for colouring
Heap pass	Gives a valid schedule quickly with a clear priority rule
DP improvement	Finds value the greedy pass missed, without ever lowering it
Colouring and mapping	Handles which room, and only changes the schedule when the change is valid
Verification	Checks the result with code that shares nothing with the allocator

17. System Architecture and Data Model

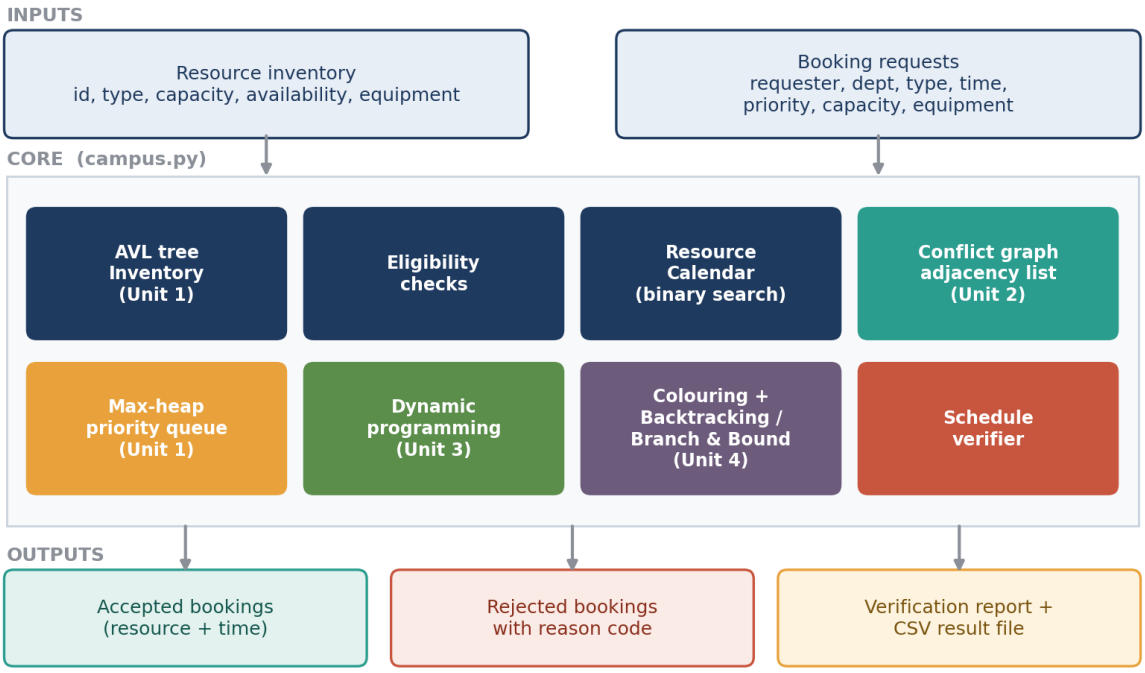


Figure 3 - Architecture. Unit numbers show which part of the course each component comes from.

17.1 Relationship model

```
Inventory (AVL tree by resource ID) --- holds ---> Resource
Resource ----------- has one -------------------> Calendar (sorted accepted bookings)
BookingRequest ----- wants a --------------------> resource TYPE (or one preferred Resource)
BookingRequest ----- is suitable for ------------> 0..m Resources (checked by rules)
BookingRequest <---- clashes with ---------------> BookingRequest (Conflict graph edge)
```

17.2 Example records (fictional sample data)

Resource	Type	Capacity	Equipment	Availability
CR-101	classroom	60	projector, whiteboard	open all day
LAB-CS2	lab	40	computers	closed Wed 09:00-13:00
LAB-CHEM	lab	24	fume_hood	out of service
GRD-1	sports ground	200	goalposts	open 06:00-19:00

#	Requester / dept	Type	When	Pri.	Needs
1	Prof. R. Sharma / CSE	classroom	Mon 09:00-10:00	9	55 seats, projector
13	Student Council / Student Affairs	seminar hall	Tue 09:00-13:00	6	200 seats, stage
21	Hostel Council / Hostel Affairs	sports ground	Wed 19:00-21:00	4	40 people

18. Algorithms, Code and Pseudocode

18.1 Availability check on a resource calendar

Each physical resource has a sorted list of bookings. A binary search finds the first booking starting at or after the new start; only that booking and the one before it can overlap, because stored bookings never overlap each other. The check is O(log b); inserting is O(b) because list elements shift.

```
def blocking(self, rid, s, e):
    """The existing booking that overlaps [s, e) on this resource, or None if the slot is free."""
    lst = self.slots.get(rid, [])
    i = bisect_left(lst, (s,))
    if i > 0 and lst[i - 1][1] > s: return lst[i - 1]
    if i < len(lst) and lst[i][0] < e: return lst[i]
    return None
```

18.2 Eligibility

```
eligibility(inventory, request):
    if end <= start:                    return INVALID_TIME
    candidates = preferred resource, or all resources of the type
    if none:                            return NO_RESOURCE_TYPE / BAD_PREFERENCE
    suitable = [r for r in candidates if no rule fails for r]
    if suitable:                        return suitable   (smallest capacity first)
    return reason of the closest candidate
```

18.3 Conflict graph by sweep

```
for each resource type:
    sort requests by start time
    for i in requests:
        for j after i:
            if start[j] >= end[i]:  break          # no later request can overlap i
            if suitable[i] and suitable[j] share a resource:
                add edge (i, j)
```

18.4 Heap pass

```
def _heap_pass(reqs, elig, assign):
    """Serve unassigned requests highest priority first; take the first (best-fit) free eligible resource."""
    by_id = {r.id: r for r in reqs}
    cal = _calendar(assign, by_id)
    h = RequestHeap()
    for r in reqs:
        if assign[r.id] is None: h.push(r)
    while not h.empty():
        r = h.pop()
        for rid in elig[r.id]:
            if cal.blocking(rid, r.start, r.end) is None:
                cal.book(rid, r.start, r.end, r.id); assign[r.id] = rid; break
```

18.5 Dynamic programming and colouring

See Sections 11.13 and 11.15 for the DP and the colour-to-room mapping code.

18.6 Top-level allocation

```
allocate_campus(inventory, requests):
    eligibility for every request         -> suitable, or rejected with reason
    conflict graph over suitable requests
    heap pass                              -> first schedule
    repeat: for each resource, DP over (its bookings + unplaced requests that fit it)
            apply only if the total value strictly rises      -> improved schedule
    colour flexible bookings; map colours to rooms if valid and no extra rooms
    heap pass again                        -> fill freed slots
    for every unplaced request: TIME_CLASH reason from its accepted neighbours
    return accepted, rejected, stage values, colouring report
```

19. Complexity and Performance Considerations

Component	Typical complexity	Why it matters
BST search	O(log n) average; O(n) worst	Degrades on sorted IDs (height 8,000 measured)
AVL search / insert	O(log n) guaranteed	Predictable lookup as the inventory grows
Tree traversal	O(n)	Used for sorted listing
Heap peek / push / pop	O(1) / O(log n) / O(log n)	Priority serving; whole pass O(n log n)
Calendar check	O(log b); insert O(b)	Fast refusal of overlaps
Eligibility of one request	O(r)	r = resources of that type
Conflict graph (sweep)	O(n log n + E)	Versus $O(n^2)$ for all pairs
Adjacency list space	O(V + E)	Avoids $O(V^2)$ cells
DP on one resource	O(m log m)	Applied repeatedly until no resource improves
Welch-Powell	$O(V^2)$	V = bookings in the group
Exact colouring	Exponential worst case	Pruned by the bound; only for groups of 14 or fewer
Schedule verifier	$O(b^2)$ per resource	Deliberately simple so it is easy to trust

19.1 Benchmark results (synthetic data)

These results come from the benchmark, which uses random synthetic requests from a seeded generator. They measure how fast and how good the algorithms are. They are not results of allocating real campus resources. Counts, heights and values are repeatable; millisecond timings vary between machines and runs.

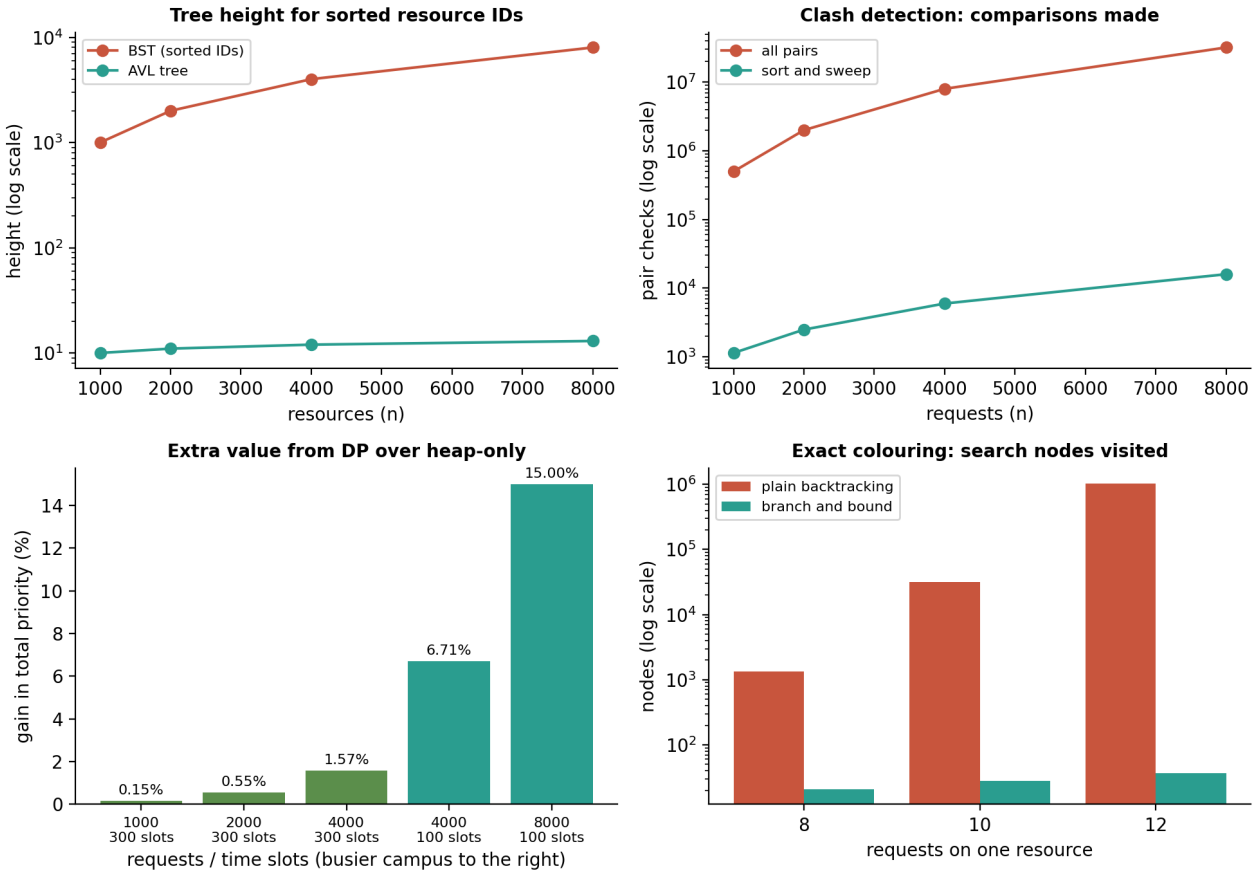


Figure 4 - Benchmark on synthetic data: tree height, clash-detection comparisons, DP gain and colouring search size.

Clash detection: 40 resources, 400 time slots (both methods build the same graph)

Requests	Clashes found	All-pairs checks	Sweep checks	All-pairs time (ms)	Sweep time (ms)
1,000	136	499,500	1,135	77.7	1.1
2,000	486	1,999,000	2,485	332.4	2.5
4,000	1,963	7,998,000	5,962	1,448.5	7.2
8,000	7,934	31,996,000	15,933	5,391.1	17.2

At 8,000 requests the sweep makes about 2,000 times fewer comparisons. Timings are from one run on the machine used to prepare this document.

Exact colouring (average of 30 seeds)

Requests on one resource	Plain backtracking (nodes)	Branch and bound (nodes)	Optimum rooms (average)
8	1,345	21	3.20
10	31,608	28	3.23
12	1,035,812	37	3.30

Both searches return the same optimum; the bound only removes wasted work.

19.2 Evaluation still planned

Real-data evaluation, comparison of the AVL inventory with a hash table, and timing with real booking volumes are planned for the final review. No claim about real-world speed or acceptance rate is made.

20. Worked Example: The Sample Campus

The sample campus is **fictional**: 13 resources and 24 requests over Monday, Tuesday and Wednesday, written to exercise every rule. The figures below are the real output of `python demo.py`: **13 of 24 requests accepted**, 11 rejected. The heap alone reached value 77; dynamic programming raised it to 81 (the auditorium re-plan, 6 to 10); the final value is 81 out of 141 requested. The independent verifier found no double booking and no rule violation.

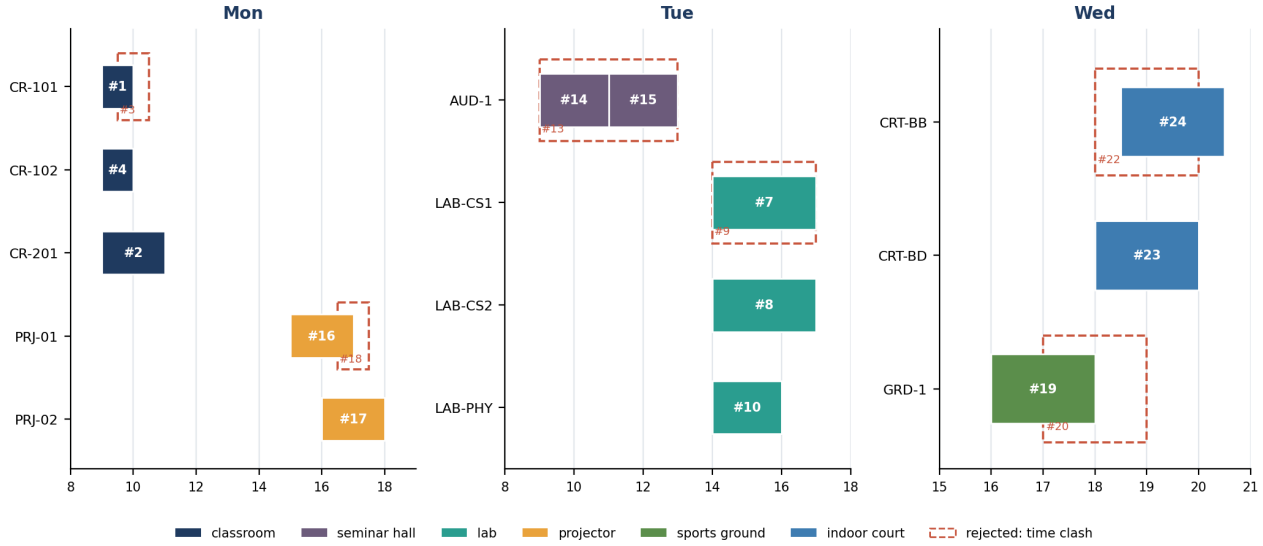


Figure 5 - Accepted bookings (filled) and requests rejected for a time clash (dashed red) on the sample campus.

20.1 Accepted bookings

#	Requester / department	Resource	When	Pri.
14	Alumni Cell (Alumni Relations)	AUD-1	Tue 09:00-11:00	5
15	Placement Cell (Training & Placement)	AUD-1	Tue 11:00-13:00	5
1	Prof. R. Sharma (CSE)	CR-101	Mon 09:00-10:00	9
4	Ms. S. Iyer (English)	CR-102	Mon 09:00-10:00	4
2	Dr. A. Khan (ECE)	CR-201	Mon 09:00-11:00	8
24	Inter-college Meet (Sports Committee)	CRT-BB	Wed 18:30-20:30	9
23	Badminton Club (Sports Committee)	CRT-BD	Wed 18:00-20:00	5
19	Football Team (Sports Committee)	GRD-1	Wed 16:00-18:00	7
7	Dr. M. Rao (CSE)	LAB-CS1	Tue 14:00-17:00	8
8	Dr. N. Pillai (IT)	LAB-CS2	Tue 14:00-17:00	8
10	Dr. V. Menon (Physics)	LAB-PHY	Tue 14:00-16:00	6
16	Photography Club (Student Clubs)	PRJ-01	Mon 15:00-17:00	3
17	Debate Society (Student Clubs)	PRJ-02	Mon 16:00-18:00	4

20.2 Rejected bookings and reasons

#	Requester	When	Reason
3	Dr. P. Nair	Mon 09:30-10:30	**TIME_CLASH** - Time clash: CR-101 is booked Mon 09:00-10:00 by #1 (CSE, priority 9); all 2 suitable classroom resources are taken in this period
5	Prof. T. Das	Mon 11:00-12:00	**CAPACITY** - Insufficient capacity: CR-201 holds 120, needs 150 (3 classroom resources checked)
6	Dr. L. Fernandes	Mon 10:00-11:00	**EQUIPMENT** - Required equipment not available: CR-201 lacks smartboard (3 classroom resources checked)

#	Requester	When	Reason
9	Mr. K. Joshi	Tue 14:00-17:00	**TIME_CLASH** - Time clash: LAB-CS1 is booked Tue 14:00-17:00 by #7 (CSE, priority 8); all 2 suitable lab resources are taken in this period
11	Dr. H. Bose	Tue 10:00-12:00	**UNAVAILABLE** - Resource unavailable: LAB-CHEM is out of service (4 lab resources checked)
12	Mr. D. Singh	Wed 10:00-12:00	**UNAVAILABLE** - Resource unavailable: LAB-CS2 is closed for maintenance Wed 09:00-13:00 (1 lab resource checked)
13	Student Council	Tue 09:00-13:00	**TIME_CLASH** - Time clash: AUD-1 is booked Tue 09:00-11:00 by #14 (Alumni Relations, priority 5); the bookings kept there are worth more together (10 vs 6)
18	Dance Club	Mon 16:30-17:30	**TIME_CLASH** - Time clash: PRJ-02 is booked Mon 16:00-18:00 by #17 (Student Clubs, priority 4); all 2 suitable projector resources are taken in this period
20	Athletics Squad	Wed 17:00-19:00	**TIME_CLASH** - Time clash: GRD-1 is booked Wed 16:00-18:00 by #19 (Sports Committee, priority 7)
21	Hostel Council	Wed 19:00-21:00	**OUTSIDE_HOURS** - Outside operating hours: GRD-1 is open 06:00-19:00 only (1 sports ground resource checked)
22	Basketball Club	Wed 18:00-20:00	**TIME_CLASH** - Time clash: CRT-BB is booked Wed 18:30-20:30 by #24 (Sports Committee, priority 9)

Reading the results. Request #3 was refused because #1 (priority 9) holds CR-101 and #2 holds CR-201. Request #13 (priority 6) was refused although it outranks the two talks #14 and #15, because they are worth 10 together; the reason text says so. Request #22 lost the basketball court to the inter-college match #24 (priority 9), while the badminton club #23 simply moved to the smaller hall CRT-BD.

20.3 Online booking on top of the schedule

`CampusBookingSystem` approves or rejects one request at a time. In the demo, a new class on CR-101 at 09:00-10:00 is rejected (it overlaps #1), while one at 10:00-11:00 on the same room is accepted because the intervals only touch. A lab workshop on Thursday is accepted on LAB-CS1, and a 400-seat auditorium request is rejected for capacity (AUD-1 holds 300).

21. Testing and Verification

The suite has **59 checks**; the last run passed all of them. Checks run on fixed seeds, so results are repeatable.

Area	Checks	What is verified
Algorithms (original)	12	AVL balance and order; BST degeneration; heap order; sweep equals all-pairs graph; greedy is clash-free; DP equals brute force; DP selection valid; exact colouring equals maximum overlap; Welch-Powell is a proper colouring.
Time helpers, compatibility	2	Clock conversion round trip; the original three-field Resource still works.
Inventory	2	Lookup through the AVL tree; duplicate IDs refused.
Overlapping and non-overlapping bookings	6	Overlap on one resource rejected; touching intervals accepted; same time on two rooms accepted; cancel frees a slot; calendar boundaries; double booking refused.
Capacity	2	Exact fit allowed; best-fit choice; demand above every room rejected with CAPACITY.
Equipment	2	Only rooms with the equipment are suitable; the missing item is named in the reason.
Availability and validity	8	Out of service; before opening; after closing; ending exactly at closing allowed; maintenance blackout; invalid time; unknown type; wrong preferred type.
Allocation correctness	6	Higher priority wins; DP beats greedy (10 vs 6); reason explains the loss; large request gets the large room; every request accepted or rejected, never both; duplicate IDs refused.

Area	Checks	What is verified
Sample campus	7	Exact accepted set and rejection codes; independent verification passes; every rejection explained; DP lifts the value; conflict graph pairs; the verifier catches broken schedules.
Pinned requests	1	Final value equals the independent per-resource DP optimum (20 seeds).
Stress	3	40 random campuses of 60 requests: schedule always valid; value never below heap-only; every request accepted or rejected with a reason.
Colour-to-room mapping	4	Valid mapping produced; refused with no common room; refused when the room is taken; applied mappings never use extra rooms.
Half-open overlap	4	[0,5) and [5,10) are not simultaneous; one-slot overlap counts; touching chain; simultaneous starts after an end.

Testing the tests. Five deliberate bugs were injected into a copy of the code (touching bookings treated as clashes, a capacity off-by-one, equipment ignored, DP disabled, overlap counting that ignored interval ends). The tests failed or crashed on every one. **Note on max_overlap():** it was reviewed for the half-open rule and already treated touching intervals correctly, so only its ordering was made explicit and regression tests were added.

22. Work Completed During Month 2

The following work is completed and forms the evidence base for Review 2:

- Chose the AVL tree for the inventory after comparing it with a plain BST, and implemented both.
- Implemented a resource inventory with capacity, availability, opening hours, maintenance periods and equipment.
- Implemented booking requests with requester, department, time, priority, capacity and equipment needs.
- Implemented per-resource calendars that refuse overlapping bookings, with half-open intervals.
- Implemented the conflict graph with two build methods and verified they agree.
- Implemented the max-heap priority queue and heap-based allocation.
- Implemented dynamic programming for the best set of bookings and combined it with the heap pass.
- Implemented Welch-Powell and exact backtracking colouring with branch and bound, and validated colour-to-room mapping.
- Implemented reason codes for every rejection, an independent verifier and an online booking class.
- Wrote 59 automated checks, a demo on sample data and a benchmark on synthetic data.
- Studied Unit 3 (Dynamic Programming) and Unit 4 (backtracking, branch and bound) and decided which topics apply.

22.1 What has NOT been claimed as completed

- No real campus data has been used; all campus data is fictional sample data.
- There is no user interface, database, login or CSV import.
- There are no recurring bookings, no set-up time between bookings, and no booking of two resources together.
- No fairness rule between departments has been implemented.
- The DP stage is not claimed to be globally optimal when requests can use several rooms.
- No real-world speed or acceptance-rate result is claimed; benchmark figures are from synthetic data.
- Graph traversal, shortest-path, spanning-tree and Unit 5 topics are not applied.

23. Evidence and Supporting Material

The 3-page PBL Progress Report – II is the formal Review 2 summary. This detailed document sits behind that report as evidence.

Evidence item	What it demonstrates
campus.py	All data structures, algorithms and the booking workflow
sample_data.py	Fictional sample inventory and requests, clearly labelled
tests.py and outputs/tests_output.txt	59 checks and their last run
demo.py, outputs/demo_output.txt, sample_allocation_result.csv	The full workflow and its result on the sample campus
bench.py, outputs/benchmark_output.txt, benchmark_results.csv	Algorithm performance on synthetic data
README.md	Setup, commands, features, limitations
Review 1 and Review 2 progress reports; Moodle grade page	Progress and faculty feedback
Unit PPTs (Units 1 to 3)	Course topics used for the DSA mapping

23.1 Evidence placement

For submission the 3-page report remains the concise official document. This documentation is attached as supporting evidence. Screenshots of the three scripts' console output should be added to the repository's `screenshots/` folder.

24. Challenges Faced and How They Were Addressed

Challenge	Why it was difficult	How it was addressed
Relating DSA to a real problem	It was easier to describe booking than to find real algorithmic operations	Mapped each structure to one operation: lookup, clash modelling, priority, best set, room counting.
Defining a clash	Bookings can overlap partly or only touch	Half-open intervals, so only real overlaps clash; covered by tests, including max_overlap().
Avoiding forced DSA usage	The syllabus has many structures that are not equally relevant	Documented each topic as Used, Considered or Not used with a reason (Sections 4 and 11).
Greedy vs optimal	At normal load DP and greedy differed by only 0.15% to 1.57%	Added a busy-campus case where the gap is 15.00%, and a sample case where DP clearly wins.
Exponential colouring search	Plain backtracking needed about 1,035,812 nodes at 12 requests	Added the branch and bound cut: 37 nodes, same answer.
Flexible requests	With several possible rooms, per-resource DP is not globally optimal	Applied DP only when the value rises; proved optimality for pinned requests; stated the limit.
Colouring may pick an invalid room	A colour class may need a room that lacks capacity or equipment	Map colours to rooms only when one room suits every booking in the class.
Keeping claims honest	Sample data can look like real results	Labelled all data as fictional; separated benchmark results from campus results.

25. Individual Contribution and Tools

This is an individual assignment, and the author, Aarohi Yadav, is the only student on the project. The work covers: the problem statement and scope; the literature review of the two papers; the study of DSA-II Units 1 to 4 and the Used / Considered / Not used decisions; the design of the allocation pipeline; the implementation, tests and benchmark in Python; and the Review 1 and Review 2 reports and this documentation.

Tools used. The code, tests and this document were prepared with the assistance of an AI tool (Claude, by Anthropic). The author is responsible for understanding and explaining the design and code, and for following the college's rules on AI assistance in submitted work.

26. Current Status and Progress

The reported overall progress is **65%**. Review 1 closed at 25% with no code. The additional 40% is the working prototype, its tests and measurements. The remaining 35% is data, interface, extra booking rules and final evaluation.

Work area	Status	Evidence in this documentation
Problem understanding	Completed	Sections 3 and 4
Stakeholder and requirements analysis	Completed	Sections 9 and 10
Existing practice study	Completed at conceptual level	Section 5
Two-paper literature review and gaps	Completed	Sections 6 and 7
Unit 1 Trees and Heaps	Implemented and tested	Sections 11, 12 and 21
Unit 2 Graphs	Implemented and tested (BFS/DFS and path algorithms not applied)	Sections 11 and 13
Unit 3 Dynamic Programming	Implemented and tested	Sections 11 and 14
Unit 4 Backtracking, Branch and Bound	Implemented and tested	Sections 11 and 13
Integrated allocation workflow	Implemented on sample data	Sections 15 to 18 and 20
Testing	Completed: 59 checks	Section 21
Benchmark	Completed on synthetic data	Section 19
Real campus data, interface, recurring bookings, fairness	Not completed	Section 27

27. Plan for Final Review

1. Replace the fictional sample data with a real timetable and booking file read from CSV, if the college can supply one.
2. Add a simple command-line interface to enter, cancel and view bookings.
3. Add recurring bookings and bookings that need two resources together, such as a room plus a portable projector.
4. Try 0/1 Knapsack or the unit-budget resource allocation DP for equipment-hour budgets and compare it with the current allocator.
5. Compare the AVL inventory with a hash table for lookups.
6. Add a fairness rule between departments and compare it with pure priority.
7. Model travel time between buildings, which would bring in shortest-path algorithms.
8. Prepare final diagrams, screenshots and documentation, and measure the system against the Review 1 objectives.

28. Limitations

- Sample data only; no real data import.
- Flexible requests: the DP stage is a per-resource improvement, not a proven global optimum.
- Priority is a single integer; no fairness, quotas or aging.
- No recurring bookings, no set-up time, no multi-resource bookings.
- Times are minutes within one week; holidays and multi-week calendars are not modelled.
- No persistence and no concurrency safety.
- The colouring step keeps the same number of rooms on the sample campus.

29. Conclusion

Review 2 turned the conceptual design of Review 1 into a working, tested prototype. The work began from the observation that registers and spreadsheets cannot keep up with growing booking demand, and that a useful system must respect capacity, equipment and availability and must explain its refusals.

From the DSA perspective, the project concluded that an AVL tree suits the ordered inventory, a sorted calendar suits instant overlap checks, a graph with an adjacency list suits the many-to-many competition between requests, a max-heap suits priority serving, dynamic programming suits choosing the best set of bookings, and backtracking with branch and bound suits exact room counts. On synthetic data, balancing cuts tree height from 8,000 to 13, the sweep cuts clash comparisons by about 2,000 times, DP adds up to 15% more value on a busy campus, and the bound cuts the colouring search from about a million nodes to 37. On the fictional sample campus, 13 of 24 requests are accepted without a single double booking, and each of the 11 rejections says why.

Binary trees, threaded trees, Huffman coding, the adjacency matrix, graph traversal and path algorithms, and Unit 5 topics were deliberately not used, and the reasons are documented. The next step is to connect the prototype to real data and a user interface and to evaluate it against the Review 1 objectives.

30. References

- *Evaluating Graph Coloring Algorithms for University Course Timetabling: Welch-Powell and Backtracking Approaches* (2025).
- *The optimization model and empirical analysis of teaching resource allocation and business collaboration under the background of higher education informatization* (2026).
- DSA-II (CCSE0301) lecture slides, Unit 1 (Trees), Unit 2 (Graphs) and Unit 3 (Dynamic Programming), Department of CSE/IT, NIET Greater Noida.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, *Introduction to Algorithms*, Prentice Hall of India (listed in the course slides).
- E. Horowitz and S. Sahni, *Fundamentals of Computer Algorithms* (listed in the course slides).
- Intelligent Campus Resources Optimization System — PBL Progress Report I and II, Data Structure and Algorithms - II (CCSE0301).
- Project repository: github.com/Aarohi591/intelligent-campus-resources-optimization-system-DSA (README, source code, tests, demo and benchmark).

Source discipline: The two research papers motivate the literature and research-gap discussion. DSA selections such as the AVL tree, max-heap, dynamic programming and the conflict graph are project design decisions derived from the required operations, not claims that the reviewed papers prescribed those structures.